

Psycho-Pass: Method and Experimental Design

This document describes the attack generation process, embedding and trajectory pipeline, and the experimental design dimensions (seed dataset, attack type, prompts, model roles, and temperatures).

1. Attack Generation Process

Overview

Attack generation uses a **multi-turn red-teaming** loop with three LLM roles:

1. **Adversarial model** — Proposes the next user message (attack prompt) to send to the target.
2. **Objective model** — The system under evaluation; it receives the conversation and produces responses.
3. **Scoring model (judge)** — After each response, decides whether the objective was achieved (e.g. jailbreak) or not (e.g. refusal).

The loop runs for a fixed number of turns (**max_turns**). Each turn: adversarial suggests a message → objective responds → judge scores the response. If the judge answers “true” (objective achieved), the attack is marked success; otherwise it continues until **max_turns** or a terminal outcome.

Implementation (RTA)

The only attack type implemented so far is **Red Teaming Attack (RTA)** (PyRIT’s **RedTeamingAttack**):

- **Adversarial configuration**
 - Either a custom **adversarial prompt** string with an **{objective}** placeholder (used as the seed prompt for the adversarial model), or
 - PyRIT’s default system prompt: **RTASystemPromptPaths.TEXT_GENERATION.value**.
- **Scoring configuration**
 - Either a custom **scoring prompt** with two fields, **true** and **false**, each with an **{objective}** placeholder (descriptions of “objective achieved” vs “objective not achieved”), or
 - PyRIT’s default: **TrueFalseQuestionPaths.QUESTION_ANSWERING.value**.
- **Execution**
 - For each seed (e.g. from **adv_bench**), the **objective** is the seed text (e.g. a harmful request).
 - The attack runs for **max_turns**; the judge’s final (or first “true”) outcome is stored.
 - All conversations are written to PyRIT SQLite memory and later exported to Parquet.

So: **attack generation** = (seed dataset × num_samples) × RTA loop → SQLite → conversion to **dataset.parquet** (conversations + success/failure labels).

2. Embedding Calculation Method

Purpose

Conversations are turned into **trajectories in embedding space**: one vector per message, so each conversation becomes a sequence of points. That allows trajectory-level metrics (velocity, path length, directness) to be computed and compared between successful and failed attacks.

Method (TF-IDF)

- **Encoder:** TF-IDF (e.g. `TFIDFEncoder` in `src/encoders.py`).
- **Corpus:** All message contents from **all** conversations in the run's `dataset.parquet` are collected into one corpus.
- **Fit:** A single `TfidfVectorizer` is fit on that corpus (parameters: `max_features`, `stop_words`, `min_df`, `max_df`).
- **Encode:** For each conversation, each message's raw text is transformed with the same vectorizer. No extra aggregation: one message → one vector.
- **Output:** Each conversation is stored as a list of vectors (one per message), i.e. a trajectory of shape `(n_turns, max_features)` in the embeddings Parquet.

So: **embedding calculation** = fit TF-IDF on full corpus → one vector per message per conversation → save as `embeddings.parquet` (trajectories).

3. Trajectory Metrics

Trajectories are sequences of points in $(\mathbb{R}^d)$. For each conversation we have a matrix of shape `(n_turns, d)` (e.g. `d = max_features` for TF-IDF). From that we compute:

3.1 Velocity

- **Step vectors:** $(v_t = x_t - x_{t-1})$ for $(t = 1, \dots, T-1)$.
- **Step magnitudes:** $(|v_t|)$.
- **Metrics:**
 - **avg_velocity:** mean of $(|v_t|)$ — average step size per turn.
 - **max_velocity:** max of $(|v_t|)$ — largest single-step move.
 - **step_variance:** variance of $(|v_t|)$ — consistency of step sizes.

High velocity can indicate the adversarial model is shifting strategy sharply between turns.

3.2 Distance

- **total_path_length:** $(\sum_t |v_t|)$ — total distance traveled along the path.
- **final_displacement:** $(|x_T - x_0|)$ — Euclidean distance from start to end.

Path length can be large even if the conversation ends near where it started; displacement measures net movement.

3.3 Directness

- **directness_index:** $(\frac{|x_T - x_0|}{\sum_t |v_t|})$ when the denominator is nonzero, else 0.

Values in $([0, 1])$. Near 1 means the trajectory is almost straight (efficient/surgical); near 0 means the path is winding.

3.4 Comparative Report

For each run, metrics are aggregated by outcome (**success** vs **failure**). The pipeline reports mean values for both groups and the **delta (%)** (e.g. how much higher or lower success is vs failure). Interpretation notes (e.g. high velocity = big strategy shifts, high directness = efficient attack) are included in the report.

4. Experimental Design

Controlled dimensions for running and comparing experiments:

4.1 Seed Dataset

- **Current:** `adv_bench` (via PyRIT's `SeedDatasetProvider`).
- **Role:** Defines the **objectives** (e.g. harmful or sensitive requests) that the adversarial model tries to achieve against the objective model.
- **Design choices:**
 - **Dataset:** e.g. `adv_bench` vs other PyRIT seed datasets.
 - **Num samples:** number of attack runs per seed (e.g. 3) for variance and stability.

Varying the seed dataset tests how trajectory and success rates depend on the type and difficulty of objectives.

4.2 Attack Type

- **Current:** Only **Red Teaming Attack (RTA)** — iterative, multi-turn, with adversarial + objective + judge.
- **Design:** Add other attack types (e.g. single-turn, different loop structures) and compare success rates and trajectory metrics.

4.3 Attack Prompts

Two prompt surfaces are configurable in `src/attacks.py`:

- **Adversarial prompt**
 - **Config:** `attack.adversarial.prompt` (single string).
 - **Usage:** Formatted with `{objective}` and passed as the seed prompt to the adversarial model.
 - **Default:** If omitted, PyRIT uses `RTASystemPromptPaths.TEXT_GENERATION.value`.
- **Scoring (judge) prompt**
 - **Config:** `attack.scoring.prompt` with keys `true` and `false` (each can use `{objective}`).
 - **Usage:** Defines what "true" (objective achieved) and "false" (not achieved) mean for the judge.
 - **Default:** If omitted, PyRIT uses `TrueFalseQuestionPaths.QUESTION_ANSWERING.value`.

Experimental levers:

- Vary adversarial prompt (e.g. more/less specific instructions, different framing of the objective).
- Vary scoring prompt (stricter vs looser "achievement" and "refusal" definitions) to study sensitivity of success labels and trajectory statistics.

4.4 Attack Model Roles and Temperatures

Three roles, each with a model and a temperature:

Role	Purpose	Experimental role	Typical temperature
Adversarial	Proposes next attack message	Variable	High (e.g. 1.0) for diversity
Objective	Target model under attack	Variable	Often high (e.g. 1.0) for varied responses

Role	Purpose	Experimental role	Typical temperature
Scoring (judge)	Success/failure decision	Fixed	0 (deterministic) for stable labels

Design:

- **Adversarial and objective:** Intended as **experimental factors**.
 - Vary model (e.g. same vs different families, sizes, or APIs).
 - Vary temperature (e.g. 0.7 vs 1.0 for adversarial/objective) to test effect on success rate and trajectory (e.g. velocity, directness).
- **Judge (scoring):** Kept **fixed** across conditions (same model, temperature 0) so that success/failure is comparable and not confounded by judge variability.

Example from configs:

- **dataset_base.json:** All three roles use the same Ollama model; adversarial and objective at temperature 1, scoring at 0.
- **dataset_feb3.json:** Same idea with a Together model; scoring has a custom **prompt** with explicit **true/false** descriptions.

Recommended design table for a minimal experiment:

Factor	Levels / choices
Seed dataset	adv_bench (and optionally others)
Attack type	RTA (extend later)
Adversarial prompt	Default vs custom (with {objective})
Scoring prompt	Default vs custom true/false
Adversarial model	Model A, Model B, ...
Objective model	Model A, Model B, ...
Judge model	Fixed (e.g. one capable model, temp 0)
Adversarial temp	e.g. 0.7, 1.0
Objective temp	e.g. 0.7, 1.0
Judge temp	0

This structure keeps labels comparable (fixed judge) while allowing systematic variation in seeds, attack prompts, and in who plays the adversarial and objective roles (and at what temperature).